

Divide Aí: entregável de SEO e GEO

Aplicativo: <https://divideai.pangeia.cloud>

Repositório: github.com/joaquimkhuck/Divideai

Time: Joaquim Huck, Estevão Antunes, Artur Bresser

1. O documento: title, meta description e H1

O que escrevemos

Title (50 caracteres)

Divide Aí: divida a conta do restaurante pela foto

Meta description (150 caracteres)

Fotografe a conta do restaurante e o Divide Aí separa quem comeu o quê. Cada um paga só o que consumiu, com taxa e couvert no cálculo, e a soma fecha.

H1

Divida a conta do restaurante pela foto

Por que assim

O que estava no ar antes era o texto que o Replit gera sozinho: `<title>Divide Aí</title>` e a description `"Divide Aí – built on Replit. Update this description to reflect the app."`. Ou seja, a página se apresentava pelo nome da ferramenta que a construiu, não pelo problema que resolve. E não existia H1 nenhum.

As três peças foram escritas com a mesma intenção e sem repetir palavra à toa:

- **Title:** marca na frente porque "Divide Aí" é o termo que alguém digita quando já ouviu falar do app, e a promessa logo depois porque quem ainda não ouviu busca pela dor ("dividir conta restaurante"). Cabe nos 60 caracteres que o Google mostra, então não é cortado com reticências.
- **Meta description:** não é fator de ranqueamento, é anúncio. Ela existe para ganhar o clique de quem já viu o resultado, então carrega o diferencial que o concorrente não tem (taxa de serviço e couvert rateados, e a soma que fecha) em vez de repetir o title.

- **H1**: descritivo, com a intenção de busca inteira em cinco palavras. Não usamos a marca como H1 porque "Divide Aí" não diz nada a quem chega de uma busca por "como dividir a conta do rodízio".

Também no HTML, por causa do GEO

Além das três peças pedidas, a home passou a servir no HTML estático um bloco "Como funciona" (H2 + os cinco passos) e um "Por que a conta fecha", mais um `application/ld+json` do tipo `WebApplication` e uma imagem de compartilhamento (`og:image`, 1200x630). O motivo é o mesmo do H1: para um crawler que não executa JavaScript, a página inteira era um título e um slogan. Agora ela tem hierarquia de heading, conteúdo e dado estruturado. O React continua substituindo o bloco ao montar.

Onde está aplicado

Peça	Arquivo	Evidência
Title, meta description, canonical, Open Graph, <code>lang="pt-BR"</code>	<code>artifacts/divide-ai/index.html</code>	<code>curl -s https://divideai.pangeia.cloud/ grep -i "<title> description"</code>
H1 na tela	<code>artifacts/divide-ai/src/pages/home.tsx</code>	visível na home, abaixo do visor da câmera
H1 no HTML servido	<code>artifacts/divide-ai/index.html</code>	<code>curl -s https://divideai.pangeia.cloud/ grep "<h1"</code>

O detalhe que a aula de GEO expõe

O Divide Aí é uma SPA em React. O HTML que o servidor entrega é praticamente `<div id="root"></div>`: o conteúdo só nasce depois que o JavaScript roda. O Googlebot renderiza JavaScript e enxergaria o H1 mesmo assim, mas os crawlers que alimentam busca generativa em geral não renderizam: GPTBot, PerplexityBot e ClaudeBot leem o HTML cru. Para eles, a home não tinha H1 nem texto, e o app simplesmente não existia como resposta possível.

Por isso o H1 e a frase de apoio foram colocados também dentro do `#root` no HTML estático. O React substitui esse bloco quando monta, então o usuário não vê diferença, e quem lê sem executar JavaScript passa a receber a página com conteúdo. É a diferença entre estar indexável e ser citável.

2. Os links

Arquivo	URL	Situação antes
robots.txt	https://divideai.pangeia.cloud/robots.txt	Existia, com duas linhas (User-agent: * e Allow: /) e nenhuma referência ao sitemap
sitemap.xml	https://divideai.pangeia.cloud/sitemap.xml	Não existia. A URL devolvia HTTP 200 com o index.html , por causa do fallback de SPA do Caddy
llms.txt	https://divideai.pangeia.cloud/llms.txt	Não existia, mesmo caso

O detalhe do fallback vale registrar: como o servidor responde index.html para qualquer caminho desconhecido, os dois arquivos ausentes retornavam 200 em vez de 404. Uma verificação por status code diria que estava tudo certo. Só olhando o corpo da resposta dá para ver que não existiam.

robots.txt

Escolha central: a única página com conteúdo público é a home. O resto (/revisar , /quem-comeu , /role/:id , /perfil , /creditos , /pagamento/) são telas de aplicação, com dado de sessão e nada de indexável. Elas estão bloqueadas, e não por preciosismo: página de app sem conteúdo próprio diluiria o que o buscador entende como sendo o site.

Os crawlers de IA aparecem nomeados e liberados na home de propósito. Essa é a decisão de GEO do projeto: queremos que o app possa ser citado como resposta quando alguém perguntar a um assistente como dividir a conta do restaurante.

Duas sutilezas que a primeira versão do arquivo errou:

- **Um grupo nomeado não herda nada do grupo ***. No protocolo, o crawler obedece ao grupo mais específico que casa com ele, e ignora o resto. Escrever os Disallow só em User-agent: * deixava GPTBot, ClaudeBot e PerplexityBot livres para rastrear /perfil , /pagamento/ e as outras telas de app. A lista agora se repete em cada grupo, de propósito.
- **Google-Extended e Applebot-Extended não são crawlers de busca**. São tokens de opt-in para uso do conteúdo em treino de modelo. Quem rastreia para o AI Overviews é o Googlebot comum. Eles estão no arquivo com essa distinção escrita em comentário, para não passar a impressão de que liberá-los faz o site aparecer em resposta.

O arquivo termina apontando o sitemap, que é como o crawler o encontra sem depender do Search Console.

sitemap.xml

Uma URL: a home. Poderíamos ter listado as quinze rotas do app para o arquivo parecer robusto, mas sitemap não é inventário de rotas, é declaração do que merece ser indexado. Listar tela de fluxo contradiria o próprio robots.txt.

A primeira versão listava também `/entrar`, e isso era um erro verificável com um `curl`: como o servidor devolve o mesmo `index.html` para qualquer rota, a página `/entrar` carrega o canonical apontando para a home. O sitemap dizia "indexe /entrar" enquanto a própria página dizia "/entrar é cópia da home". Ficou só a home.

llms.txt

É o formato proposto em llms.txt.org para dar ao modelo de linguagem uma versão limpa do que o site é, em Markdown, sem menu nem JavaScript. Enquanto o sitemap responde "quais páginas existem", o llms.txt responde "o que este produto é e o que é verdade sobre ele".

O nosso carrega os fatos que um modelo precisa para responder sobre o app sem errar:

- o fluxo em cinco passos, de foto até cobrança;
- que item compartilhado é dividido em centavos inteiros, com diferença máxima de um centavo entre as partes;
- que taxa de serviço e couvert são rateados na proporção do consumo, pelo método do maior resto, e não em partes iguais;
- que a soma dos valores individuais sempre fecha com o total;
- que só a leitura da foto consome crédito;
- e, deliberadamente, o que o produto **não** é: não é vaquinha nem controle de despesa recorrente.

Essa última linha existe porque a falha típica de um modelo é aproximar o produto do vizinho mais famoso. Dizer o que ele não é reduz a chance de ele ser descrito errado.

3. As próximas 5 features com score

O framework

RICE: $(\text{Reach} \times \text{Impact} \times \text{Confidence}) \div \text{Effort}$. As unidades foram fixadas antes de pontuar qualquer coisa, senão o score vira opinião com cara de número:

- **Reach:** pessoas alcançadas a cada 100 rolês. O PRD fala em grupos de 3 a 8 pessoas; assumimos 4 por mesa, dentro dessa faixa, então 100 rolês são cerca de 400 pessoas: 1 fotografia e 3 são cobradas em cada uma. Alcançada é quem toca a tela da feature ou recebe o resultado dela, não a mesa inteira por tabela.
- **Impact:** 3 massivo, 2 alto, 1 médio, 0,5 baixo.

- **Confidence:** 100% quando existe código ou diff pronto, 80% quando o caminho técnico é claro, 50% quando depende de comportamento que nunca medimos.
- **Effort:** semanas de uma pessoa.

As candidatas saíram do PRD, dos mockups já desenhados e do histórico do git, não de brainstorm. Cada linha abaixo tem uma fonte no repositório.

A tabela

#	Feature	R	I	C	E	RICE
1	Pix de verdade: BR Code com o valor da parte	300	2	50%	0,5	600
2	Link público do rolê para quem foi cobrado	300	2	80%	1	480
3	IA sugerir o que é item compartilhado	100	1	80%	0,5	160
4	Convidado marca o próprio consumo pelo celular dele	300	3	50%	3	150
5	Cache da leitura por hash da imagem	20	1	100%	0,25	80

Uma a uma

3. IA sugerir o que é item compartilhado (160)

Hoje todo item nasce "SEM DONO" e o botão de avançar só libera quando 100% foi atribuído. Couvert, entrada e porção são quase sempre da mesa inteira, e alguém marca isso na mão toda vez. O prompt de extração passaria a devolver um sinal de "provavelmente compartilhado" e o app pré-marcaria.

Custa meio dia, porque o campo entra no JSON que a IA já devolve. Fica em terceiro por uma correção de honestidade no Reach: a tela de atribuição é tocada só por quem fotografou, uma pessoa por rolê, e não pelas quatro da mesa. Enquanto a feature 4 não existir, o alcance dela é 100, não 400.

1. Pix de verdade: BR Code com o valor da parte (600)

O campo de Pix copia só a chave: o valor ainda é digitado à mão no banco, que é exatamente onde o erro de centavo reaparece depois de todo o cuidado do motor de cálculo. Pior: quem não tem conta cobra com `divideai@pix.com.br`, uma chave fictícia que está no código como fallback.

O payload EMV é gerado no próprio cliente, sem backend. Confiança 50% porque não sabemos quantas contas têm chave cadastrada. Hoje o PostHog captura só pageview automático: responder isso exige instrumentar um evento no cadastro da chave, que é o próximo passo depois desta entrega.

2. Link público do rolê para quem foi cobrado (480)

O PRD tem como P0: "como quem foi cobrado, quero ver o que devo e marcar como pago". Foi entregue pela metade. Hoje a cobrança é um texto aberto no WhatsApp, e a única pessoa que enxerga o rolê é a dona do cookie. O próprio código admite o buraco: um comentário em `payments.ts` registra que o pagador nunca tem o cookie do dono.

4. Convidado marca o próprio consumo (150)

Estrategicamente é a maior da lista, e o RICE a coloca em quarto. Vale explicar por quê, em vez de esconder: impacto 3 (é a etapa mais longa do fluxo e o único mecanismo de crescimento viral que o PRD prevê), mas esforço 3 semanas e confiança 50%, porque exige token público, endpoint sem cookie de dono, tela nova e resolver duas pessoas editando o mesmo rolê ao mesmo tempo.

RICE é bom para ordenar trabalho incremental e ruim para apostas estruturais: ele divide por esforço, então toda aposta grande desce na lista. Registramos o número e a ressalva. Se a decisão do time for priorizar crescimento em vez de eficiência, esta sobe, e a justificativa é essa, não o score.

5. Cache da leitura por hash da imagem (80)

Não é feature nova, é regressão. O commit `be0e349` criou o cache e um sync posterior do Replit o apagou. Hoje cada leitura repetida chama a IA de novo, gasta um crédito do usuário e US\$ 0,014 nossos. Confiança 100% porque o diff existe no histórico. Score baixo porque o alcance é pequeno, e ainda assim entra: é um quarto de semana e devolve crédito cobrado indevidamente.

O que o score não decide

Duas coisas ficaram fora da tabela de propósito, porque não são features, são decisões:

- **Login obrigatório para escanear.** O commit `c8d96b1` passou a exigir conta antes da primeira foto. O PRD, o README e o `replit.md` prometem, os três, primeiro resultado sem cadastro. Isso atinge 100% de quem chega, no primeiro toque, e precisa de decisão do time, não de priorização.
- **Chaves de desenvolvimento do Clerk em produção.** O console do app publicado avisa a cada carregamento. É dívida de deploy, não escopo de produto.

A honestidade do exercício

Os números de Reach desta tabela são estimativa fundamentada, não medição: até hoje o produto não tinha um único evento instrumentado. O PostHog que entra nesta entrega registra pageview, o que já responde quantas pessoas chegam e onde param. As três métricas do PRD (tempo da foto até o valor, itens que não precisaram de correção, contas que chegam ao pagamento) exigem eventos próprios, que ainda não existem. Ou seja: esta tabela é a última que se defende por argumento. A próxima se defende por dado, e a diferença entre as duas é o trabalho de instrumentar.

4. A conta do PostHog

Conta criada em posthog.com (login por GitHub), projeto na nuvem dos Estados Unidos.

O código está no app publicado: `posthog-js` inicializado em `artifacts/divide-ai/src/main.tsx`, com a configuração isolada em `artifacts/divide-ai/src/lib/analytics.ts`. A captura de pageview usa o pacote de defaults `2025-05-24`, que registra navegação por History API. Isso importa aqui porque a aplicação é uma SPA com wouter: sem isso, o PostHog contaria uma única visita e ignoraria toda a troca de telas dentro do fluxo.

O token de projeto (`phc_...`) fica no código do cliente porque é público por definição: ele só permite escrever evento, nunca ler dado. As chaves que não podem vaziar (Anthropic, Stripe, Clerk) continuam apenas nas variáveis de ambiente do servidor.

Evidência: `docs/evidencias/posthog-eventos.jpg`, painel de Activity filtrado por `Library = web`, ou seja, só o que veio de navegador de verdade, com os `Pageview` e `Pageleave` de `divideai.pangeia.cloud`. O filtro está no print de propósito: durante o diagnóstico do bloqueador disparamos algumas requisições de teste por linha de comando, e elas não são visita de ninguém.

O bloqueador de rastreador: o erro que quase passou batido

Na primeira versão, o app chamava `us.i.posthog.com` direto, como manda a documentação. O código estava certo, a chave estava certa, o PostHog inicializava no navegador (sessão criada, configuração remota carregada) e mesmo assim o painel continuava com zero evento.

O que o teste mostrou:

Chamada	Pelo terminal	Pelo navegador
POST <code>us.i.posthog.com/i/v0/e/</code> (captura)	200 <code>{"status":"Ok"}</code>	Failed to fetch
POST <code>us.i.posthog.com/decide/</code>	200	200
GET <code>us-assets.i.posthog.com/.../config.js</code>	200	200

Ou seja: o bloqueador de rastreador instalado no navegador deixava passar a configuração e cortava só a captura. Do lado do servidor estava tudo funcionando, e nenhum erro aparecia no console.

A correção é a que o próprio PostHog recomenda: um proxy reverso no nosso domínio. O Caddy passou a repassar `divideai.pangeia.cloud/ingest` para a nuvem do PostHog, e o cliente aponta para `/ingest`. Como a requisição sai da mesma origem do site, não existe domínio de terceiro para a lista de bloqueio reconhecer.

Depois da mudança, o `$pageview` disparado pelo próprio `posthog-js` aparece na aba de rede como `POST /ingest/i/v0/e/` com status 200, e os eventos aparecem no painel.

Isso não é detalhe de bastidor. Não temos número próprio de quantos usuários nossos usam bloqueador, e não vamos citar estimativa de terceiro sem fonte; o que sabemos é o que medimos: no navegador em que o teste foi feito, 100% dos eventos se perdiam em silêncio. Sem o proxy, a métrica do produto seria menor que a realidade por uma margem desconhecida, sem nenhum erro avisando.

Por que isso era o buraco mais sério do projeto

O PRD define três métricas de acompanhamento: tempo da foto até o valor por pessoa, porcentagem de itens que não precisaram de correção e porcentagem de contas que chegam até o pagamento. Nenhuma delas tinha um único evento instrumentado no código. Com analytics no ar, a próxima priorização deixa de ser palpite.

Validação técnica

- `pnpm --filter @workspace/divide-ai run typecheck`: sem erros
- `pnpm --filter @workspace/divide-ai run build`: build de produção concluído, com `robots.txt`, `sitemap.xml` e `llms.txt` presentes em `dist/public`
- Deploy no Coolify a partir da `main`, em <https://divideai.pangeia.cloud>
- Verificação em produção: `title`, `meta description`, `canonical` e `lang="pt-BR"` no HTML servido; `H1` presente no HTML cru (sem executar JavaScript); os três arquivos respondendo 200 com o `content-type` correto (`text/plain` e `text/xml`); `posthog-js` no bundle e evento chegando ao painel